



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Kali Linux хранит информацию о каждом процессе, чтобы помочь сохранить порядок в системе, и каждому процессу присваивается номер, называемый идентификатором процесса (PID).

Оболочка Linux также вводит понятие заданий для облегчения рабочего процесса пользователя во время терминальной сессии. Например, `cat error.txt | wc -m` – отобразит два процесса, которые оболочка считает одним заданием. В Linux есть возможность выборочно приостанавливать выполнение заданий и возобновлять их выполнение в более позднее время. Этого можно достичь с помощью специальных команд, которые мы далее рассмотрим.

Фоновые процессы (bg)

В предыдущих уроках команды выполнялись на переднем плане, что означает, что терминал занят и никакие другие команды не могут быть выполнены, пока не завершится текущее задание. Поскольку большинство наших примеров были короткими и понятными, это не вызвало проблем. Однако в последующих модулях мы будем выполнять более длинные и сложные команды, которые мы можем отправить в фоновый режим, чтобы вернуть контроль над терминалом и выполнить дополнительные команды. Самый быстрый способ отправить процесс в фоновый режим – это добавить амперсанд (&) в конец команды, чтобы отправить ее в фоновый режим сразу после запуска. Давайте рассмотрим небольшой пример:

```
kali@kali:~$ ping -c 400 localhost > ping.txt &
```

В этом примере мы отправили 400 эхо-запросов ICMP на локальный интерфейс с помощью команды `ping` и записали результаты в файл `ping.txt`. Выполнение автоматически происходит в фоновом режиме, оставляя оболочку свободной для дополнительных операций.

Но что бы произошло, если бы мы не добавили амперсанд в конце команды? Команда была бы запущена на переднем плане, и мы были бы вынуждены либо ждать, пока команда завершится, чтобы вернуть контроль над терминалом, либо остановить команду с помощью `Ctrl + C`.

Другой вариант - приостановить задание с помощью `Ctrl + Z` после того, как оно уже запущено. После приостановки задания мы можем возобновить его выполнение в фоновом режиме с помощью команды `bg`:

```
kali@kali:~$ ping -c 400 localhost > ping.txt
```

```
^Z
[1]+  Stopped ping -c 400 localhost > ping.txt
kali@kali:~$ bg
[1]+  ping -c 400 localhost > ping.txt
kali@kali:~$
```

Теперь задание выполняется в фоновом режиме, и мы можем продолжать использовать терминал по своему усмотрению. При выполнении этого задания помните, что некоторые процессы чувствительны ко времени и могут дать неверные результаты, если их приостановить надолго.

jobs и fg

Чтобы быстро проверить состояние наших эхо-запросов ICMP, нам нужно использовать две дополнительные команды: `jobs` и `fg`.

Встроенная утилита `jobs` выводит список заданий, выполняемых в текущем терминальном сеансе, а команда `fg` возвращает задание на передний план:

```
kali@kali:~$ ping -c 400 localhost > ping.txt
^Z
kali@kali:~$ find / -name sbd.exe
^Z
kali@kali:~$ jobs
kali@kali:~$ fg %1
```

Управление процессами: ps и kill

Одной из самых полезных команд для мониторинга процессов является команда `ps`. В отличие от команды `jobs`, `ps` выводит список процессов по всей системе, а не только для текущего терминального сеанса. Эта утилита считается стандартом для Unix-подобных ОС.

Для пентестера, после получения удаленного доступа к системе одним из первых пунктов проверки является выяснение того, какое программное обеспечение в настоящее время запущено на взломанной машине. Это

может помочь нам повысить наши привилегии или собрать дополнительную информацию для получения дальнейшего доступа в сеть.

В качестве примера запустим Wireshark, а затем попытаемся найти идентификатор его процесса (PID) из командной строки с помощью команды ps:

```
kali@kali:~$ ps -ef
```

-e – ключ, позволяющий выбрать все процессы

-f – ключ, выводящий полную информацию (UID, PID, PPID и т.д.)

Найти наше приложение в этом огромном списке определенно непросто, но поскольку мы знаем имя искомого приложения, мы можем заменить ключ -e на -C:

```
kali@kali:~$ ps -fC wireshark
```

Допустим, мы хотим остановить процесс Wireshark, не взаимодействуя с графическим интерфейсом. Здесь нам может помочь команда kill:

```
kali@kali:~$ sudo kill 1245
```

Утилита kill в Linux используется для отправки сигналов процессам. Хотя её название подразумевает завершение процессов, она может посылать разные типы сигналов, не только для завершения. Команда kill работает с процессами, используя их идентификаторы (PID). По умолчанию kill посылает сигнал TERM (15), который запрашивает корректное завершение процесса. Если процесс не отвечает на сигнал TERM, можно использовать сигнал KILL, который принудительно завершает процесс:

```
kali@kali:~$ sudo kill -9 1245
```

Об остальных ключах утилиты подробнее можно прочитать на странице документации утилиты

```
kali@kali:~$ man kill
```

pgkill

Утилита pgkill в Linux используется для завершения процессов по их именам или другим атрибутам, таким как пользователь, терминал или сигнал. В отличие от команды kill, которая требует указания PID (идентификатора процесса), pgkill позволяет завершать процессы по более удобным и человекочитаемым критериям. Это делает pgkill особенно полезной для ситуаций, когда необходимо завершить множество процессов, принадлежащих одному пользователю или соответствующих определенному шаблону имени.

Первым аргументом она принимает имя программы

```
kali@kali:~$ sudo pgkill airodump
```

Используя ключ -u можно завершить программы, запущенные конкретным пользователем, в примере ниже пользователя user

```
kali@kali:~$ sudo pgkill -u user
```

Команды, используемые в уроке:

```
kali@kali:~$ ping -c 400 localhost > ping.txt &
```

```
kali@kali:~$ ping -c 400 localhost > ping.txt
```

```
kali@kali:~$ bg
```

```
kali@kali:~$ find / -name sbd.exe
```

```
kali@kali:~$ jobs
```

```
kali@kali:~$ fg %1
```

```
kali@kali:~$ ps -ef
```

```
kali@kali:~$ ps -fC wireshark
```

```
kali@kali:~$ sudo kill 1245
```

```
kali@kali:~$ sudo pgkill airodump
```

```
kali@kali:~$ sudo pgkill -u user
```